

HỌC VIỆN KỸ THUẬT QUÂN SỰ
VIỆN CÔNG NGHỆ THÔNG TIN & TRUYỀN THÔNG
CHƯƠNG TRÌNH THẠC SĨ

BÁO CÁO BÀI TẬP LỚN
KẾT THÚC MÔN HỌC

Môn học: Khai phá Dữ liệu (Data Mining)

Đề tài:

Nghiên cứu và cải tiến thuật toán BFSPMiner
khai phá mẫu tuần tự trên luồng dữ liệu

Sinh viên thực hiện: Thân Thị Hằng
Lê Thị Kiều Trang
Hà Tuấn Hiếu

Giáo viên hướng dẫn: TS Nguyễn Hoàng Sinh

Hà Nội, 2026

Mục lục

1	Giới thiệu	5
1.1	Đặt vấn đề	5
1.2	Hạn chế của các phương pháp hiện tại	5
1.3	Mục tiêu nghiên cứu	6
1.4	Cấu trúc báo cáo	6
2	Cơ sở lý thuyết và phát biểu bài toán	7
2.1	Các khái niệm cơ bản	7
2.1.1	Luồng dữ liệu	7
2.1.2	Mẫu tuần tự	7
2.1.3	Độ hỗ trợ, độ tin cậy và độ nâng	7
2.1.4	Cây Inverted T_0	8
2.2	Phát biểu bài toán	8
2.3	Dữ liệu đầu vào và đầu ra	9
2.3.1	Đầu vào	9
2.3.2	Đầu ra	9
2.4	Tổng quan các thuật toán liên quan	10
3	Thuật toán BFSPMiner gốc	11
3.1	Tổng quan kiến trúc	11
3.2	Bước 1: Xây dựng cây Inverted T_0	11
3.3	Bước 2: Cắt tỉa SSBE	12
3.4	Bước 3: Trích xuất mẫu phổ biến	13
3.5	Bước 4: Dự đoán item tiếp theo	13
3.6	Mình họa chi tiết từng bước	14
3.6.1	Bước 1: Nhận item a ($t = 0$)	14
3.6.2	Bước 2: Nhận item b ($t = 1$)	15
3.6.3	Bước 3: Nhận item a ($t = 2$)	15
3.6.4	Bước 4: Nhận item b ($t = 3$)	16
3.6.5	Bước 5: Nhận item c ($t = 4$)	16
3.6.6	Trích xuất mẫu phổ biến ($\text{min_support} = 0.2$)	17
4	Các cải tiến đề xuất	18
4.1	Cải tiến 1: Adaptive maxPatternLength	18
4.1.1	Vấn đề	18
4.1.2	Giải pháp đề xuất	18

4.1.3	Minh họa hoạt động	19
4.2	Cải tiến 2: Gap-Aware Episode Mining	20
4.2.1	Vấn đề	20
4.2.2	Giải pháp đề xuất	20
4.2.3	Minh họa hoạt động	21
4.3	Phân tích độ phức tạp	22
5	Thực nghiệm và kết quả	23
5.1	Tập dữ liệu	23
5.2	Cấu hình thực nghiệm	23
5.3	Kết quả tổng hợp	24
5.4	Tính hoàn chỉnh của tập mẫu	25
5.5	Chất lượng dự đoán trực tuyến	25
5.6	Nghiên cứu cắt bỏ (Ablation Study)	26
5.6.1	Tác động của Adaptive maxPatternLength	26
5.6.2	Tác động của Gap-Aware Episode Mining	27
5.7	Phân tích khả năng mở rộng	28
6	Hướng dẫn cài đặt và sử dụng	29
6.1	Yêu cầu hệ thống	29
6.2	Cài đặt	29
6.3	Chạy Demo cơ bản	29
6.4	Chạy thực nghiệm đầy đủ	30
6.5	Đối chiếu với Java Baseline	30
6.6	Unit Testing	30
6.7	Cấu trúc mã nguồn	30
7	Kết luận và hướng phát triển	31
7.1	Kết luận	31
7.2	Đóng góp chính	31
7.3	Hướng phát triển	32
A	Mã nguồn chính	33
A.1	core/config.py — Cấu hình tham số	33
A.2	core/inverted_t0_tree.py — Cay Inverted T0	33
A.3	core/bfspminer.py — Thuật toán chính	34
A.4	core/adaptive_max_length.py — Adaptive Length	35
A.5	core/episode_extension.py — Gap-Aware Mining	36

Danh sách hình vẽ

5.1	So sánh Precision@3 giữa bốn phương pháp trên REDD và MSNBC	24
5.2	Số lượng mẫu phổ biến theo kích thước luồng trên REDD và MSNBC . . .	25
5.3	Bộ nhớ tiêu thụ theo cấu hình L_{max} : Fixed-12 gấp 17× Adaptive	26
5.4	Tác động của max_gap trên MSNBC: bước nhảy vọt tại $g = 2$	27
5.5	Thời gian chạy theo kích thước luồng: khả năng mở rộng tuyến tính	28

Danh sách bảng

2.1	Dữ liệu đầu vào minh họa	9
2.2	Đầu ra: Tập mẫu tuần tự phổ biến	9
2.3	So sánh các thuật toán khai phá mẫu tuần tự	10
3.1	Kết quả trích xuất mẫu từ cây ($\text{min_support} = 0.2$)	17
4.1	Quá trình thích ứng L_{max} trên MSNBC	19
4.2	Quá trình thích ứng L_{max} trên REDD	19
4.3	So sánh mẫu sinh ra giữa consecutive và gap-aware	22
4.4	Phân tích độ phức tạp thuật toán	22
5.1	Thông tin tập dữ liệu thực nghiệm	23
5.2	Kết quả tổng hợp trên toàn bộ luồng dữ liệu	24
5.3	Số lượng mẫu phổ biến theo kích thước luồng	25
5.4	Precision@3 theo kích thước luồng	25
5.5	Tác động của Adaptive trên MSNBC (100K sự kiện)	26
5.6	Tác động của Gap Constraint trên MSNBC (100K sự kiện)	27
5.7	Thời gian chạy và bộ nhớ theo kích thước luồng — REDD	28
6.1	Cấu trúc thư mục mã nguồn	30

Chương 1

Giới thiệu

1.1 Đặt vấn đề

Khai phá mẫu tuần tự (Sequential Pattern Mining – SPM) trên luồng dữ liệu liên tục là một trong những bài toán quan trọng bậc nhất trong lĩnh vực khai phá dữ liệu hiện đại. Trong thời đại Internet vạn vật (IoT), thương mại điện tử và hệ thống thông tin quy mô lớn, dữ liệu không còn được lưu trữ dưới dạng tĩnh mà liên tục phát sinh dưới dạng *luồng* (stream) — một chuỗi vô hạn các sự kiện đến với tốc độ cao và phân bố thay đổi theo thời gian.

Các ứng dụng thực tế đòi hỏi khả năng khai phá mẫu tuần tự trực tuyến bao gồm:

- **Giám sát IoT:** Phát hiện các chuỗi hành vi bất thường từ cảm biến (ví dụ: tín hiệu tiêu thụ điện năng bất thường trong nhà thông minh).
- **Phân tích hành vi web:** Khai phá các chuỗi tương tác của người dùng trên website (clickstream) để tối ưu trải nghiệm và hệ thống gợi ý.
- **An ninh mạng:** Phát hiện các chuỗi sự kiện tấn công trong log hệ thống.
- **Tin sinh học:** Phân tích chuỗi DNA, protein theo thời gian thực.

Luồng dữ liệu mang ba đặc trưng thách thức khiến các thuật toán khai phá truyền thống (PrefixSpan [?], SPADE [?]) không thể áp dụng trực tiếp:

1. **Vô hạn và liên tục:** Không thể lưu toàn bộ dữ liệu vào bộ nhớ.
2. **Tốc độ cao:** Mỗi sự kiện cần được xử lý ngay lập tức (single-pass).
3. **Phân bố thay đổi:** Các mẫu hành vi có thể thay đổi theo thời gian (concept drift).

1.2 Hạn chế của các phương pháp hiện tại

Phần lớn các thuật toán khai phá mẫu tuần tự trên luồng dữ liệu dựa trên mô hình xử lý theo lô (*batch-based*), chia luồng vô hạn thành các khối có kích thước cố định rồi khai phá từng khối. Cách tiếp cận này tồn tại hai vấn đề nghiêm trọng:

1. **Mất mẫu qua ranh giới lô:** Các chuỗi mẫu nằm xuyên suốt nhiều batch bị bỏ sót hoàn toàn, dẫn đến kết quả khai phá không đầy đủ.

2. **Cắt tĩa cục bộ sai lầm:** Quyết định loại bỏ trong từng batch có thể xóa các mẫu có tần suất cao trên toàn cục nhưng xuất hiện rải rác ở các batch khác nhau.

Thuật toán BFSPMiner (Batch-Free Sequential Pattern Miner) được đề xuất bởi Hasani et al. [?] đã khắc phục các nhược điểm trên bằng khung khai phá *batch-free* (không chia lô). Tuy nhiên, bản gốc vẫn tồn tại hai hạn chế:

- Tham số chiều dài mẫu tối đa (`maxPatternLength`) cố định.
- Chỉ hỗ trợ mẫu liên tiếp (consecutive), bỏ qua các phụ thuộc hành vi bị gián đoạn.

1.3 Mục tiêu nghiên cứu

Bài tập lớn này thực hiện ba mục tiêu chính:

1. **Nghiên cứu và cài đặt** thuật toán BFSPMiner gốc bằng Python, tối ưu cấu trúc dữ liệu so với bản Java.
2. **Đề xuất hai cải tiến:**
 - *Adaptive maxPatternLength*: Tự động điều chỉnh chiều dài mẫu tối đa dựa trên entropy luồng và áp lực bộ nhớ.
 - *Gap-Aware Episode Mining*: Mở rộng quá trình sinh mẫu để phát hiện các phụ thuộc hành vi không liên tiếp.
3. **Đánh giá thực nghiệm** trên hai tập dữ liệu thực tế: REDD (IoT, 458K sự kiện) và MSNBC (clickstream, 424K sự kiện).

1.4 Cấu trúc báo cáo

Báo cáo được tổ chức thành 7 chương:

- **Chương 1:** Giới thiệu bài toán, mục tiêu nghiên cứu.
- **Chương 2:** Cơ sở lý thuyết, phát biểu bài toán, dữ liệu đầu vào/đầu ra.
- **Chương 3:** Thuật toán BFSPMiner gốc với minh họa chi tiết từng bước.
- **Chương 4:** Các cải tiến đề xuất (Adaptive + Gap-Aware).
- **Chương 5:** Thực nghiệm và kết quả.
- **Chương 6:** Hướng dẫn cài đặt và sử dụng chương trình.
- **Chương 7:** Kết luận và hướng phát triển.

Chương 2

Cơ sở lý thuyết và phát biểu bài toán

2.1 Các khái niệm cơ bản

2.1.1 Luồng dữ liệu

Định nghĩa 2.1 (Luồng dữ liệu). Một luồng dữ liệu (data stream) $S = \langle s_1, s_2, s_3, \dots \rangle$ là một chuỗi vô hạn các phần tử (item) $s_i \in \Sigma$, trong đó Σ là bảng ký hiệu (alphabet) hữu hạn và mỗi s_i đến tại thời điểm t_i theo thứ tự tăng dần.

Ví dụ: Trong hệ thống giám sát năng lượng, $\Sigma = \{\text{fridge, light, washer, microwave, dishwasher}\}$ và luồng $S = \langle \text{fridge, light, fridge, washer, light, ...} \rangle$ ghi lại thứ tự các thiết bị tiêu thụ điện.

2.1.2 Mẫu tuần tự

Định nghĩa 2.2 (Mẫu tuần tự liên tiếp). Một mẫu tuần tự (sequential pattern) $p = \langle p_1, p_2, \dots, p_k \rangle$ là một chuỗi con liên tiếp (consecutive subsequence) của S nếu tồn tại vị trí i sao cho $s_i = p_1, s_{i+1} = p_2, \dots, s_{i+k-1} = p_k$.

Ví dụ: Với $S = \langle a, b, a, b, c, d \rangle$, mẫu $\langle a, b, c \rangle$ là mẫu tuần tự liên tiếp (xuất hiện tại vị trí $i = 3$), nhưng $\langle a, c \rangle$ không phải mẫu liên tiếp (vì a và c không kề nhau).

2.1.3 Độ hỗ trợ, độ tin cậy và độ nâng

Định nghĩa 2.3 (Độ hỗ trợ – Support). Cho luồng S đã quan sát n phần tử. Độ hỗ trợ của mẫu p là:

$$\text{supp}(p) = \frac{\text{count}(p)}{n} \quad (2.1)$$

trong đó $\text{count}(p)$ là số lần p xuất hiện liên tiếp trong n phần tử đầu tiên của S .

Định nghĩa 2.4 (Mẫu phổ biến – Frequent Pattern). Mẫu p được gọi là phổ biến nếu $\text{supp}(p) \geq \sigma$, với σ là ngưỡng support tối thiểu do người dùng định nghĩa.

Định nghĩa 2.5 (Độ tin cậy – Confidence). Với mẫu $p = \langle p_1, \dots, p_{k-1}, p_k \rangle$, confidence được định nghĩa:

$$\text{conf}(p) = \frac{\text{count}(p)}{\text{count}(\langle p_1, \dots, p_{k-1} \rangle)} \quad (2.2)$$

thể hiện xác suất p_k xuất hiện ngay sau tiền tố $\langle p_1, \dots, p_{k-1} \rangle$.

Định nghĩa 2.6 (Độ nâng – Lift). Lift đo mức độ tương quan giữa tiền tố và hậu tố:

$$\text{lift}(p) = \frac{\text{supp}(p)}{\text{supp}(\langle p_1, \dots, p_{k-1} \rangle) \times \text{supp}(\langle p_k \rangle)} \quad (2.3)$$

Lift > 1 cho thấy sự xuất hiện cùng nhau mang tính tích cực; Lift $= 1$ là độc lập; Lift < 1 là tiêu cực.

2.1.4 Cây Inverted T_0

Định nghĩa 2.7 (Cây Inverted T_0). Cây Inverted T_0 là một cây tiền tố (trie) lưu trữ các mẫu tuần tự theo thứ tự *đảo ngược thời gian* — sự kiện gần nhất nằm ở cấp gốc (root), sự kiện xa nhất ở cấp lá (leaf). Mỗi nút chứa:

- **value:** ký hiệu item.
- **count:** số lần xuất hiện tích lũy.
- **batch_count:** số batch mà nút đã xuất hiện.
- **tid:** batch ID tại thời điểm nút được tạo.
- **time_stamps:** danh sách thời điểm xuất hiện gần nhất.
- **children:** bảng băm (HashMap) ánh xạ ký hiệu $\rightarrow$ nút con.

Lợi thế: Việc lưu theo thứ tự đảo ngược giúp tra cứu ngữ cảnh (context lookup) hiệu quả — khi dự đoán item tiếp theo, ta bắt đầu từ sự kiện gần nhất (root level) và duyệt xuống.

2.2 Phát biểu bài toán

Bài toán (Khai phá mẫu tuần tự batch-free trên luồng dữ liệu):

Cho:

- Một luồng dữ liệu vô hạn $S = \langle s_1, s_2, \dots \rangle$ với $s_i \in \Sigma$.
- Ngưỡng support tối thiểu $\sigma > 0$.
- Chiều dài mẫu tối đa L_{max} .
- Tham số cắt tĩa: $\alpha, \varepsilon, \delta$.

Yêu cầu:

- Phát hiện **liên tục** (online) tất cả các mẫu tuần tự phổ biến p (với $|p| \leq L_{max}$ và $\text{supp}(p) \geq \sigma$).

- Quét dữ liệu **một lượt** (single-pass): mỗi item chỉ được đọc đúng một lần.
- Kiểm soát bộ nhớ ở mức hằng số thông qua cắt tỉa định kỳ.
- Hỗ trợ dự đoán item tiếp theo dựa trên các mẫu đã khai phá.

2.3 Dữ liệu đầu vào và đầu ra

Để minh họa rõ ràng bài toán, chúng tôi trình bày ví dụ cụ thể với dữ liệu đầu vào và đầu ra tương ứng.

2.3.1 Đầu vào

Bảng 2.1: Dữ liệu đầu vào minh họa

Thành phần	Giá trị
Luồng sự kiện S	$\langle a, b, a, b, c, a, b, a, b, c, d, a \rangle$
Số phần tử	$n = 12$
Bảng ký hiệu Σ	$\{a, b, c, d\}$
<code>maxPatternLength</code> L_{max}	5
Ngưỡng support σ	0.15
<code>batchLength</code>	6
<code>delta</code> δ	1000

2.3.2 Đầu ra

Bảng 2.2: Đầu ra: Tập mẫu tuần tự phổ biến

STT	Mẫu (Pattern)	Count	Support	Confidence	Loại
1	(a)	4	0.333	—	unigram
2	(b)	4	0.333	—	unigram
3	(a, b)	4	0.333	1.000	bigram
4	(c)	2	0.167	—	unigram
5	(b, c)	2	0.167	0.500	bigram
6	(a, b, c)	2	0.167	0.500	trigram

Dự đoán: Với ngữ cảnh hiện tại là hai sự kiện gần nhất $\langle \dots, a \rangle$, hệ thống dự đoán top-3 item tiếp theo là: $[c, d, a]$.

Giải thích: Mẫu (a, b) có support 0.333 và confidence 1.0, nghĩa là mỗi lần sự kiện a xuất hiện, sự kiện b *luôn* theo sau. Mẫu (a, b, c) có support 0.167, nghĩa là chuỗi $a \rightarrow b \rightarrow c$ xuất hiện $2/12 = 16.7\%$ thời gian.

2.4 Tổng quan các thuật toán liên quan

Bảng 2.3: So sánh các thuật toán khai phá mẫu tuần tự

Thuật toán	Loại	Batch-free	Gap	Adaptive
PrefixSpan [?]	Tĩnh	Không	Không	Không
SPADE [?]	Tĩnh	Không	Có	Không
GSP [?]	Tĩnh	Không	Có	Không
Batch-based SPM	Luồng	Không	Không	Không
BFSPMiner [?]	Luồng	Có	Không	Không
BFSPMiner-Improved	Luồng	Có	Có	Có

Chương 3

Thuật toán BFSPMiner gốc

Chương này trình bày chi tiết thuật toán BFSPMiner (Batch-Free Sequential Pattern Miner) được đề xuất bởi Hassani et al. [?], bao gồm kiến trúc tổng quan, các bước xử lý chính, và minh họa từng bước trên dữ liệu cụ thể.

3.1 Tổng quan kiến trúc

BFSPMiner xử lý luồng dữ liệu theo mô hình *one-pass* (một lượt) với pipeline gồm 4 bước chính cho mỗi sự kiện đến:

1. **Nhận sự kiện:** Đọc item mới từ luồng.
2. **Sinh mẫu và cập nhật cây:** Xây dựng danh sách mẫu đảo ngược, chèn vào cây Inverted T_0 .
3. **Theo dõi lô:** Đếm số item đã xử lý để xác định ranh giới batch logic.
4. **Cắt tỉa SSBE:** Định kỳ loại bỏ các nút không triển vọng khỏi cây.

Sau khi xử lý toàn bộ luồng (hoặc tại bất kỳ thời điểm nào), có thể thực hiện:

- **Trích xuất mẫu phổ biến:** Duyệt DFS cây, thu thập các mẫu có support $\geq \sigma$.
- **Dự đoán item tiếp theo:** So khớp ngữ cảnh gần nhất với các mẫu, xếp hạng theo confidence.

3.2 Bước 1: Xây dựng cây Inverted T_0

Khi nhận sự kiện mới s_t tại thời điểm t , thuật toán thực hiện:

1. Thêm s_t vào danh sách sự kiện: $events.append(s_t)$.
2. Xây dựng danh sách mẫu đảo ngược (reversed pattern list):

$$pattern_list = [s_t, s_{t-1}, s_{t-2}, \dots, s_{t-L_{max}+1}] \quad (3.1)$$

(lấy tối đa L_{max} phần tử gần nhất theo thứ tự đảo ngược).

3. Duyệt $pattern_list$ từ trái sang phải, tại mỗi bước:

- Nếu nút con tương ứng **đã tồn tại**: tăng `count` lên 1.
- Nếu nút con **chưa tồn tại**: tạo nút mới với `count = 1`.

Algorithm 1 Thêm sự kiện vào cây Inverted T_0

Require: Cây $\mathcal{T}$, sự kiện s_t , batch hiện tại b

```

1:  $\mathcal{T}.events.append(s_t)$ 
2:  $t \leftarrow |\mathcal{T}.events| - 1$ 
3:  $pattern\_list \leftarrow [s_{t-i} \mid i = 0, 1, \dots, \min(L_{max}, t + 1) - 1]$ 
4:  $visited \leftarrow \emptyset$  {Tránh đếm trùng}
5:  $node \leftarrow \mathcal{T}.root$ 
6: for  $i = 0$  to  $|pattern\_list| - 1$  do
7:    $item \leftarrow pattern\_list[i]$ 
8:   if  $item \in node.children$  then
9:      $node \leftarrow node.children[item]$ 
10:    if  $id(node) \notin visited$  then
11:       $node.count \leftarrow node.count + 1$ 
12:       $visited.add(id(node))$ 
13:    end if
14:  else
15:     $new\_node \leftarrow TreeObject(item, node, b)$ 
16:     $node.children[item] \leftarrow new\_node$ 
17:     $node \leftarrow new\_node$ 
18:     $visited.add(id(node))$ 
19:  end if
20: end for
21:  $\mathcal{T}.root.count \leftarrow |\mathcal{T}.events|$ 

```

3.3 Bước 2: Cắt tỉa SSBE

Để kiểm soát kích thước cây, BFSPMiner sử dụng chiến lược cắt tỉa SSBE (Single-Scan Batch Error). Cứ sau mỗi δ batch, thuật toán duyệt đệ quy từng nút và kiểm tra điều kiện loại bỏ an toàn:

$$c.count + b' \times (\lceil \alpha \times l \rceil - 1) \leq \varepsilon \times b_s \times l \quad (3.2)$$

trong đó:

- $c.count$: tần suất tích lũy của nút c .
- b_s : số batch kể từ khi nút c được tạo.
- b' : số batch mà c vắng mặt ($b' = b_s - c.batch_count$).
- α : tỷ lệ tăng tối đa mỗi batch (mặc định 0.05).

- ε : ngưỡng pruning (mặc định 0.1).
- l : chiều dài luồng hiện tại.

Ý nghĩa: Ngay cả trong kịch bản *lạc quan nhất* (giả sử nút xuất hiện tối đa $\lceil \alpha \times l \rceil - 1$ lần trong mỗi batch bị thiếu), nếu tần suất vẫn không vượt ngưỡng, nút đó chắc chắn không bao giờ trở thành mẫu phổ biến và được loại bỏ an toàn.

Algorithm 2 Cắt tỉa SSBE

Require: Nút n , batch hiện tại b , tham số $\alpha, \varepsilon, \delta$, chiều dài luồng l

```

1: for mỗi nút con  $c$  của  $n$  do
2:    $b_s \leftarrow b - (c.tid - c.tid \bmod \delta)$  {Số batch kể từ lần tạo}
3:    $b' \leftarrow b_s - c.batch\_count$  {Số batch vắng mặt}
4:   if  $c.count + b' \times (\lceil \alpha \times l \rceil - 1) \leq \varepsilon \times b_s \times l$  then
5:     Xóa  $c$  và toàn bộ cây con {Loại bỏ an toàn}
6:   else
7:     SSBEPRUNE( $c, b, \alpha, \varepsilon, \delta, l$ ) {Đệ quy}
8:   end if
9: end for
  
```

3.4 Bước 3: Trích xuất mẫu phổ biến

Tại bất kỳ thời điểm nào, có thể trích xuất tập mẫu phổ biến bằng cách duyệt DFS (Depth-First Search) toàn bộ cây:

1. Bắt đầu từ gốc, duyệt từng nhánh.
2. Tại mỗi nút, tính $\text{supp} = \frac{c.count}{root.count}$.
3. Nếu $\text{supp} \geq \sigma$: thu thập đường đi từ gốc đến nút, **đảo ngược** lại để được mẫu gốc.
4. Tính $\text{confidence} = \frac{c.count}{parent.count}$.

Lưu ý quan trọng: Vì cây lưu theo thứ tự đảo ngược, đường đi $root \rightarrow b \rightarrow a$ tương ứng với mẫu gốc (a, b) (sau khi đảo ngược).

3.5 Bước 4: Dự đoán item tiếp theo

Module dự đoán sử dụng trực tiếp các mẫu phổ biến đã khai phá:

1. Lấy $\text{snapshot} = L_{max}$ sự kiện gần nhất (đảo ngược).
2. Với mỗi mẫu p có chiều dài > 1 : kiểm tra xem tiền tố (đảo ngược) có khớp với đầu snapshot không.

3. Xếp hạng các mẫu khớp theo confidence giảm dần.
4. Trả về top- k phần tử cuối (target) duy nhất.
5. Nếu chưa đủ k : bổ sung từ unigram có support cao nhất (fallback).

Algorithm 3 Dự đoán phần tử tiếp theo

Require: Cây $\mathcal{T}$, số dự đoán k , ngưỡng support σ

Ensure: Danh sách k phần tử dự đoán $\hat{I}$

```

1:  $\mathcal{F} \leftarrow \mathcal{T}.\text{EXTRACTPATTERNS}(\sigma)$ 
2:  $\text{snap} \leftarrow \text{REVERSE}(\mathcal{T}.\text{events}[-L_{\max} :])$ 
3:  $\text{cands} \leftarrow \emptyset$ 
4: for mỗi mẫu  $p \in \mathcal{F}$  với  $|p| > 1$  do
5:    $\text{prefix} \leftarrow \text{REVERSE}(p[1..|p| - 1])$ 
6:   if  $\text{prefix}$  là tiền tố của  $\text{snap}$  then
7:      $\text{conf} \leftarrow \text{supp}(p)/\text{supp}(p[1..|p| - 1])$ 
8:     Thêm  $(p[|p|], \text{conf})$  vào  $\text{cands}$ 
9:   end if
10: end for
11: Sắp xếp  $\text{cands}$  theo  $\text{conf}$  giảm dần
12:  $\hat{I} \leftarrow$  top- $k$  phần tử duy nhất từ  $\text{cands}$ 
13: if  $|\hat{I}| < k$  then
14:   Bổ sung từ unigram có support cao nhất {Fallback}
15: end if
16: return  $\hat{I}$ 

```

3.6 Minh họa chi tiết từng bước

Để làm rõ cách thức hoạt động của thuật toán, chúng tôi minh họa chi tiết quá trình xử lý trên luồng dữ liệu mẫu:

$$S = \langle a, b, a, b, c \rangle, \quad L_{\max} = 3$$

3.6.1 Bước 1: Nhận item a ($t = 0$)

- $\text{events} = [a]$
- $\text{pattern_list} = [a]$ (chỉ có 1 item, đảo ngược = chính nó)
- Cập nhật cây: Tạo nút a làm con của root

Trạng thái cây sau bước 1:

```

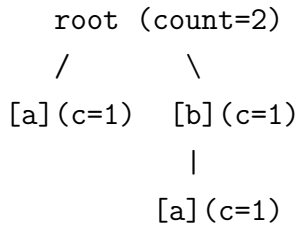
root (count=1)
|
[a] (count=1)

```

3.6.2 Bước 2: Nhận item b ($t = 1$)

- $events = [a, b]$
- $pattern_list = [b, a]$ (đảo ngược: b ở $t = 1$, a ở $t = 0$)
- Cập nhật cây:
 - Duyệt item b : tạo nút **b** làm con của root
 - Duyệt item a : tạo nút **a** làm con của nút **b**
 - Đồng thời, nút **a** (con của root) được tăng count (vì a xuất hiện trong $pattern_list$)

Trạng thái cây sau bước 2:

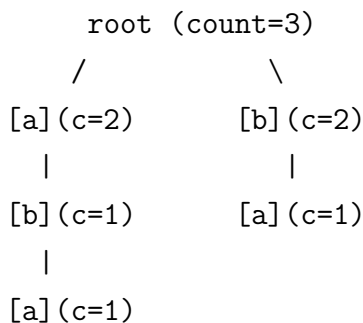


Giải thích: Đường đi $root \rightarrow b \rightarrow a$ (đảo ngược) đại diện cho mẫu (a, b) .

3.6.3 Bước 3: Nhận item a ($t = 2$)

- $events = [a, b, a]$
- $pattern_list = [a, b, a]$ (đảo ngược 3 items gần nhất: $s_2 = a$, $s_1 = b$, $s_0 = a$)
- Cập nhật cây:
 - Duyệt item a : nút **a** (con root) đã tồn tại $\rightarrow count\ 1 \rightarrow 2$
 - Duyệt item b : tạo nút **b** làm con của nút **a** (con root)
 - Duyệt item a : tạo nút **a** làm con của nút **b** (con của **a**)

Trạng thái cây sau bước 3:



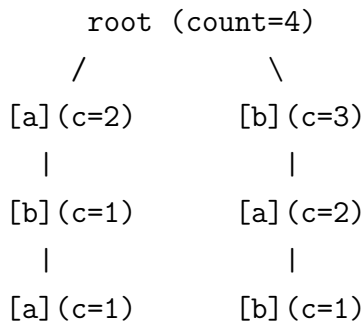
Giải thích:

- root $\rightarrow$ a: mẫu (a) — count = 2
- root $\rightarrow$ a $\rightarrow$ b: đảo ngược = (b, a) — count = 1
- root $\rightarrow$ b $\rightarrow$ a: đảo ngược = (a, b) — count = 1
- root $\rightarrow$ a $\rightarrow$ b $\rightarrow$ a: đảo ngược = (a, b, a) — count = 1

3.6.4 Bước 4: Nhận item b ($t = 3$)

- events = $[a, b, a, b]$
- pattern_list = $[b, a, b]$ ($s_3 = b, s_2 = a, s_1 = b$)
- Cập nhật cây:
 - Nút **b** (con root): count $2 \rightarrow 3$
 - Nút **a** (con **b**): count $1 \rightarrow 2$
 - Tạo nút **b** làm con của nút **a** (con **b**)

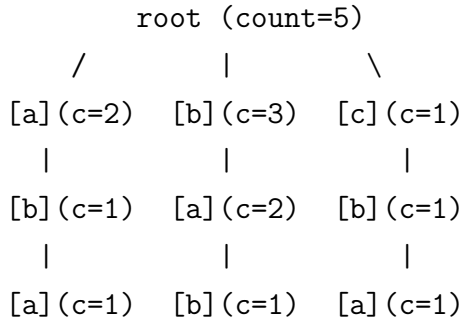
Trạng thái cây sau bước 4:



3.6.5 Bước 5: Nhận item c ($t = 4$)

- events = $[a, b, a, b, c]$
- pattern_list = $[c, b, a]$ ($s_4 = c, s_3 = b, s_2 = a$)
- Cập nhật cây:
 - Tạo nút **c** (con root): count = 1
 - Tạo nút **b** làm con của **c**: count = 1
 - Tạo nút **a** làm con của **b** (con **c**): count = 1

Trạng thái cây sau bước 5 (trạng thái cuối):



3.6.6 Trích xuất mẫu phổ biến ($\text{min_support} = 0.2$)

Duyệt DFS toàn bộ cây, đảo ngược đường đi để được mẫu gốc:

Bảng 3.1: Kết quả trích xuất mẫu từ cây ($\text{min_support} = 0.2$)

Đường đi trong cây	Mẫu gốc	Count	Support	Phổ biến?
root $\rightarrow$ a	(a)	2	0.40	✓
root $\rightarrow$ a $\rightarrow$ b	(b, a)	1	0.20	✓
root $\rightarrow$ b	(b)	3	0.60	✓
root $\rightarrow$ b $\rightarrow$ a	(a, b)	2	0.40	✓
root $\rightarrow$ b $\rightarrow$ a $\rightarrow$ b	(b, a, b)	1	0.20	✓
root $\rightarrow$ c	(c)	1	0.20	✓
root $\rightarrow$ c $\rightarrow$ b	(b, c)	1	0.20	✓
root $\rightarrow$ c $\rightarrow$ b $\rightarrow$ a	(a, b, c)	1	0.20	✓

Nhận xét: Chỉ với 5 sự kiện, cây Inverted T_0 đã nắm bắt được tất cả các mẫu tuần tự liên tiếp có ý nghĩa, bao gồm cả mẫu dài (a, b, c).

Chương 4

Các cải tiến đề xuất

Chương này trình bày hai cải tiến chính mà chúng tôi đề xuất cho thuật toán BFSP-Miner, nhằm giải quyết hai hạn chế được chính tác giả gốc thừa nhận trong phần “Hướng phát triển” (Future Work) của bài báo gốc [?].

4.1 Cải tiến 1: Adaptive maxPatternLength

4.1.1 Vấn đề

Tham số `maxPatternLength` (L_{max}) cố định của BFSPMiner gốc gây ra hai vấn đề đối lập:

- L_{max} **quá nhỏ**: Bỏ sót các mẫu dài có ý nghĩa (ví dụ: chuỗi hành vi mua hàng phức tạp).
- L_{max} **quá lớn**: Bùng nổ tổ hợp, bộ nhớ tăng theo hàm mũ (cây có tối đa $|\Sigma|^{L_{max}}$ nút).

4.1.2 Giải pháp đề xuất

Chúng tôi đề xuất cơ chế kiểm soát thích ứng (adaptive control) dựa trên hai tín hiệu:

1. **Shannon Entropy** của luồng gần đây: đo mức độ đa dạng của dữ liệu.
2. **Áp lực bộ nhớ** (RSS): giám sát bộ nhớ tiến trình real-time.

Cứ sau mỗi chu kỳ Δ sự kiện (mặc định $\Delta = 5,000$), hệ thống cập nhật:

$$L_{max}(t) = \begin{cases} \max(L_{min}, L_{max}(t-1) - 1), & \text{nếu } M_{cur} > \mu_{high} \\ \max(L_{min}, L_{max}(t-1) - 1), & \text{nếu } H > \theta_{high} \\ \min(L_{ceil}, L_{max}(t-1) + 1), & \text{nếu } H < \theta_{low} \\ L_{max}(t-1), & \text{trường hợp khác} \end{cases} \quad (4.1)$$

Với các ngưỡng mặc định: $\mu_{high} = 500$ MB, $\theta_{high} = 3.0$, $\theta_{low} = 1.5$, $L_{min} = 3$, $L_{ceil} = 15$. **Entropy Shannon** được tính trên bộ đệm $\mathcal{B}$ chứa các sự kiện trong chu kỳ hiện tại:

$$H = - \sum_{i \in \Sigma(\mathcal{B})} p_i \log_2 p_i \quad (4.2)$$

trong đó p_i là tỷ lệ xuất hiện của ký hiệu i trong $\mathcal{B}$.

Algorithm 4 Cập nhật maxPatternLength Thích ứng

Require: Sự kiện mới s_t , bộ đệm $\mathcal{B}$, chu kỳ Δ

Ensure: Giá trị L_{max} đã cập nhật

```

1:  $count \leftarrow count + 1$ 
2: Thêm  $s_t$  vào bộ đệm  $\mathcal{B}$ 
3: if  $count \bmod \Delta = 0$  then
4:    $M_{cur} \leftarrow \text{GETPROCESSMEMORY}()$  {RSS tính bằng MB}
5:    $H \leftarrow -\sum_{i \in \Sigma(\mathcal{B})} p_i \log_2 p_i$  {Entropy Shannon}
6:   if  $M_{cur} > \mu_{\text{high}}$  then
7:      $L_{max} \leftarrow \max(L_{min}, L_{max} - 1)$  {Áp lực bộ nhớ}
8:   else if  $H > \theta_{\text{high}}$  then
9:      $L_{max} \leftarrow \max(L_{min}, L_{max} - 1)$  {Luồng đa dạng}
10:  else if  $H < \theta_{\text{low}}$  then
11:     $L_{max} \leftarrow \min(L_{ceil}, L_{max} + 1)$  {Luồng lặp lại}
12:  end if
13:  Xóa  $\mathcal{B}$  {Reset bộ đệm}
14: end if
15: return  $L_{max}$ 

```

4.1.3 Minh họa hoạt động

Ví dụ trên MSNBC (17 danh mục, entropy cao):

Bảng 4.1: Quá trình thích ứng L_{max} trên MSNBC

Chu kỳ	Items đã xử lý	Entropy H	RAM (MB)	L_{max}
0	0	—	5.0	5 (khởi tạo)
1	5,000	3.42	8.2	4 (giảm: $H > 3.0$)
2	10,000	3.38	9.1	3 (giảm: $H > 3.0$)
3	15,000	3.35	9.5	3 (giữ: đã đạt L_{min})
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Ví dụ trên REDD (8 thiết bị, entropy thấp):

Bảng 4.2: Quá trình thích ứng L_{max} trên REDD

Chu kỳ	Items đã xử lý	Entropy H	RAM (MB)	L_{max}
0	0	—	5.0	5 (khởi tạo)
1	5,000	1.12	6.3	6 (tăng: $H < 1.5$)
2	10,000	1.08	6.8	7 (tăng: $H < 1.5$)
3	15,000	1.15	7.1	8 (tăng: $H < 1.5$)
$\vdots$	$\vdots$	$\vdots$	$\vdots$	$\vdots$

Kết luận: Trên luồng entropy cao (MSNBC), hệ thống tự động thu hẹp L_{max} để tiết kiệm bộ nhớ. Trên luồng entropy thấp (REDD), L_{max} được mở rộng để khai phá mẫu dài hơn. Kết quả thực nghiệm (Chương 5) cho thấy giảm $17\times$ bộ nhớ trên MSNBC.

4.2 Cải tiến 2: Gap-Aware Episode Mining

4.2.1 Vấn đề

BFSPMiner gốc chỉ hỗ trợ mẫu liên tiếp (consecutive), nghĩa là các sự kiện trong mẫu phải kề nhau trong luồng. Điều này bỏ qua một lớp lớn các phụ thuộc hành vi thực tế, khi hành vi bị gián đoạn bởi các sự kiện nhiễu.

Ví dụ thực tế: Trên website, người dùng thường thực hiện:

Xem sản phẩm A $\rightarrow$ (cuộn trang) $\rightarrow$ (xem quảng cáo) $\rightarrow$ *Thêm A vào giỏ hàng*

Mẫu ẩn “Xem $\rightarrow$ Thêm giỏ hàng” bị bỏ sót vì có 2 sự kiện xen giữa.

4.2.2 Giải pháp đề xuất

Chúng tôi mở rộng quá trình sinh mẫu để cho phép tối đa $g = \text{max_gap}$ sự kiện không liên quan xen kẽ, trong phạm vi cửa sổ $w = \text{window_size}$.

Định nghĩa 4.1 (Episode Gap-Aware). Episode $e = \langle e_1, e_2, \dots, e_k \rangle$ xảy ra trong S nếu tồn tại các chỉ số $i_1 < i_2 < \dots < i_k$ sao cho:

- $e_j = s_{i_j}$ với mọi j
- $i_{j+1} - i_j - 1 \leq g$ (gap constraint)
- $i_k - i_1 + 1 \leq w$ (window constraint)

Algorithm 5 Sinh mẫu Gap-Aware Episode**Require:** Chuỗi sự kiện $\mathcal{E}$, thời điểm hiện tại t , L_{max} , gap g , cửa sổ w **Ensure:** Tập mẫu hợp lệ $\mathcal{P}$

```

1:  $target \leftarrow \mathcal{E}[t]$ 
2:  $\mathcal{P} \leftarrow \{[target]\}$ ;  $start \leftarrow \max(0, t - w + 1)$ 
3:  $paths \leftarrow \{([target], t)\}$ ;  $seen \leftarrow \emptyset$ 
4: for  $level = 1$  to  $L_{max} - 1$  do
5:    $new\_paths \leftarrow \emptyset$ 
6:   for mỗi  $(pat, last)$  trong  $paths$  do
7:     for  $j = last - 1$  xuống  $\max(start, last - g)$  do
8:        $ext \leftarrow pat \oplus [\mathcal{E}[j]]$  {Mở rộng ngược}
9:       if  $ext \notin seen$  then
10:        Thêm  $ext$  vào  $\mathcal{P}$  và  $seen$ 
11:        Thêm  $(ext, j)$  vào  $new\_paths$ 
12:       end if
13:       if  $|new\_paths| \geq C_{max}$  then
14:         thoát vòng lặp {Giới hạn bùng nổ tổ hợp ( $C_{max} = 50$ )}
15:       end if
16:     end for
17:   end for
18:    $paths \leftarrow new\_paths$ 
19: end for
20: return  $\mathcal{P}$ 

```

4.2.3 Minh họa hoạt động**Ví dụ:** $events = [a, b, c, a, d]$, item mới = d ($t = 4$), $g = 2$, $w = 5$, $L_{max} = 3$.**Với thuật toán gốc (consecutive only):**

- $pattern_list = [d, a, c]$ (3 items liên tiếp gần nhất)
- Mẫu sinh ra: (d) , (a, d) , (c, a, d)

Với Gap-Aware ($g = 2$):

- Bắt đầu: $paths = \{([d], 4)\}$
- Level 1: quét từ $t = 3$ đến $\max(0, 4 - 2) = 2$
 - $j = 3$: $\mathcal{E}[3] = a \rightarrow ext = [d, a] \leftarrow \text{consecutive}$
 - $j = 2$: $\mathcal{E}[2] = c \rightarrow ext = [d, c] \leftarrow \text{gap} = 1$ (bỏ qua a ở vị trí 3)
- Level 2: mở rộng tiếp $[d, a]$ và $[d, c]$
 - Từ $[d, a]$ ($last = 3$): quét $j = 2, 1 \rightarrow [d, a, c], [d, a, b]$
 - Từ $[d, c]$ ($last = 2$): quét $j = 1, 0 \rightarrow [d, c, b], [d, c, a]$

So sánh kết quả:

Bảng 4.3: So sánh mẫu sinh ra giữa consecutive và gap-aware

Mẫu	Consecutive	Gap-Aware ($g = 2$)
(d)	✓	✓
(a, d)	✓	✓
(c, a, d)	✓	✓
(c, d)	×	✓ (bỏ qua a)
(b, d)	—	— (gap = 2, ngoài phạm vi $g = 2$ từ vị trí 4)
(b, a, d)	—	—
(b, c, d)	—	—

4.3 Phân tích độ phức tạp

Bảng 4.4: Phân tích độ phức tạp thuật toán

Thành phần	Thời gian (mỗi item)	Không gian
BFSPMiner gốc	$O(L_{max})$	$O(\Sigma ^{L_{max}})$
+ Adaptive	$O(L_{max} + \Delta)$ mỗi Δ items	$O(\mathcal{B})$ thêm
+ Gap-Aware	$O(L_{max} \times C_{max})$	$O(\Sigma ^{L_{max}} \times (1 + g))$
SSBE Pruning	$O(\mathcal{T})$ mỗi δ batch	—

Nhận xét: Giới hạn cứng $C_{max} = 50$ đảm bảo chi phí Gap-Aware luôn tuyến tính theo L_{max} , ngăn chặn bùng nổ tổ hợp $O(2^w)$.

Chương 5

Thực nghiệm và kết quả

5.1 Tập dữ liệu

Bảng 5.1: Thông tin tập dữ liệu thực nghiệm

Tập dữ liệu	Lĩnh vực	Số sự kiện	$ \Sigma $	Entropy	Đặc điểm
REDD [?]	IoT (năng lượng)	457,833	8	Thấp (~ 1.1)	Lặp lại cao
MSNBC [?]	Clickstream	423,776	17	Cao (~ 3.4)	Đa dạng

REDD (Reference Energy Disaggregation Dataset): Tập dữ liệu IoT ghi lại tín hiệu tiêu thụ điện năng từ 8 loại thiết bị gia đình. Đặc trưng: entropy thấp, cấu trúc lặp lại cao — đại diện cho kịch bản dễ dự đoán.

MSNBC: Tập dữ liệu clickstream ghi lại lịch sử duyệt web trên MSNBC.com, chứa 17 danh mục trang. Đặc trưng: entropy cao, hành vi đa dạng và xen kẽ — đại diện cho kịch bản thách thức nhất.

5.2 Cấu hình thực nghiệm

Phần cứng: Intel Core i7, 16GB RAM.

Phần mềm: Python 3.9+, thư viện `psutil`, `tracemalloc`.

Đo lường: Bộ nhớ đo bằng `tracemalloc` (peak heap); Thời gian đo bằng `time.perf_counter()`.

Baselines:

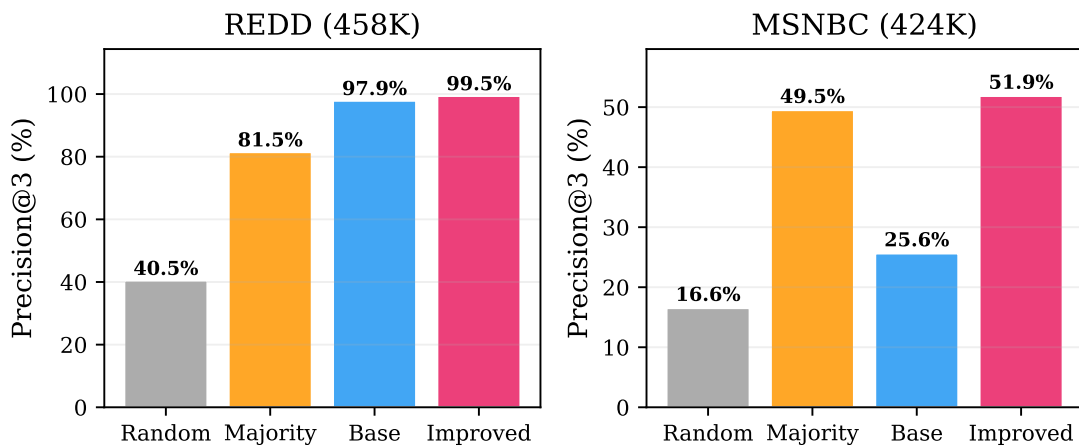
- **Naive-Random**: Dự đoán ngẫu nhiên 3 item từ tập đã quan sát.
- **Naive-Majority**: Luôn dự đoán 3 item có tần suất cao nhất.
- **BFSPMiner-Base**: Thuật toán gốc ($L_{max} = 5$, `adaptive = tắt`, `gap = tắt`).
- **BFSPMiner-Improved**: Đầy đủ (`adaptive = bật`, `gap = bật`, $g = 2$, $w = 5$).

Đánh giá dự đoán: Precision@3 trực tuyến — cứ mỗi 500 sự kiện (sau 1,000 sự kiện khởi động), dự đoán top-3 và kiểm tra. Ngưỡng $\sigma = 0.001$.

5.3 Kết quả tổng hợp

Bảng 5.2: Kết quả tổng hợp trên toàn bộ luồng dữ liệu

Phương pháp	REDD (458K)		MSNBC (424K)	
	Patterns	Prec@3	Patterns	Prec@3
Naive-Random	—	40.48%	—	16.55%
Naive-Majority	—	81.51%	—	49.53%
BFSPMiner-Base	105	97.92%	523	25.65%
BFSPMiner-Improved	203	99.45%	996	51.89%
<i>Cải thiện vs Base</i>	<i>+93%</i>	<i>+1.5pp</i>	<i>+90%</i>	<i>+26.2pp</i>



Hình 5.1: So sánh Precision@3 giữa bốn phương pháp trên REDD và MSNBC

Phân tích REDD: Do tính lặp lại cao, Naive-Majority đã đạt 81.51%. BFSPMiner-Base đạt 97.92% và Improved nâng lên 99.45%, đồng thời phát hiện thêm 93% số mẫu (105 $\rightarrow$ 203).

Phân tích MSNBC: Đây là kết quả quan trọng nhất. BFSPMiner-Base chỉ đạt 25.65% — thấp hơn cả Naive-Majority (49.53%). Điều này cho thấy rằng buộc consecutive nghiêm trọng hạn chế khả năng dự đoán trên luồng entropy cao. BFSPMiner-Improved đạt **51.89%**, vượt qua Majority Baseline, đồng thời tăng 90% số mẫu.

5.4 Tính hoàn chỉnh của tập mẫu

Bảng 5.3: Số lượng mẫu phổ biến theo kích thước luồng

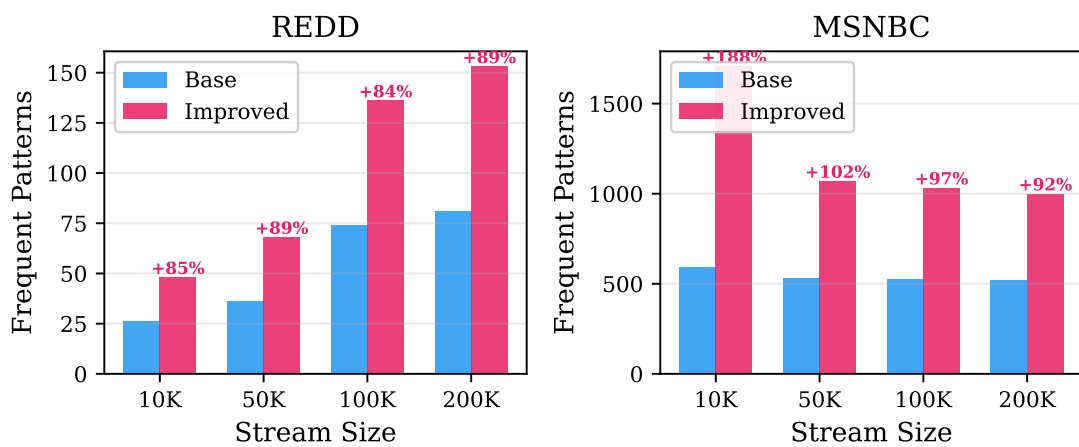
Kích thước	REDD			MSNBC		
	Base	Imp	Δ	Base	Imp	Δ
10K	26	48	+85%	592	1705	+188%
50K	36	68	+89%	529	1069	+102%
100K	74	136	+84%	525	1032	+97%
200K	81	153	+88%	520	999	+92%

BFSPMiner-Improved nhất quán phát hiện thêm **84–93%** số mẫu trên REDD và **92–188%** trên MSNBC.

5.5 Chất lượng dự đoán trực tuyến

Bảng 5.4: Precision@3 theo kích thước luồng

Kích thước	REDD		MSNBC	
	Base	Improved	Base	Improved
10K	100.0%	100.0%	27.8%	22.2%
50K	100.0%	100.0%	24.5%	38.8%
100K	97.0%	100.0%	25.8%	46.5%
200K	96.5%	99.8%	25.9%	50.5%



Hình 5.2: Số lượng mẫu phổ biến theo kích thước luồng trên REDD và MSNBC

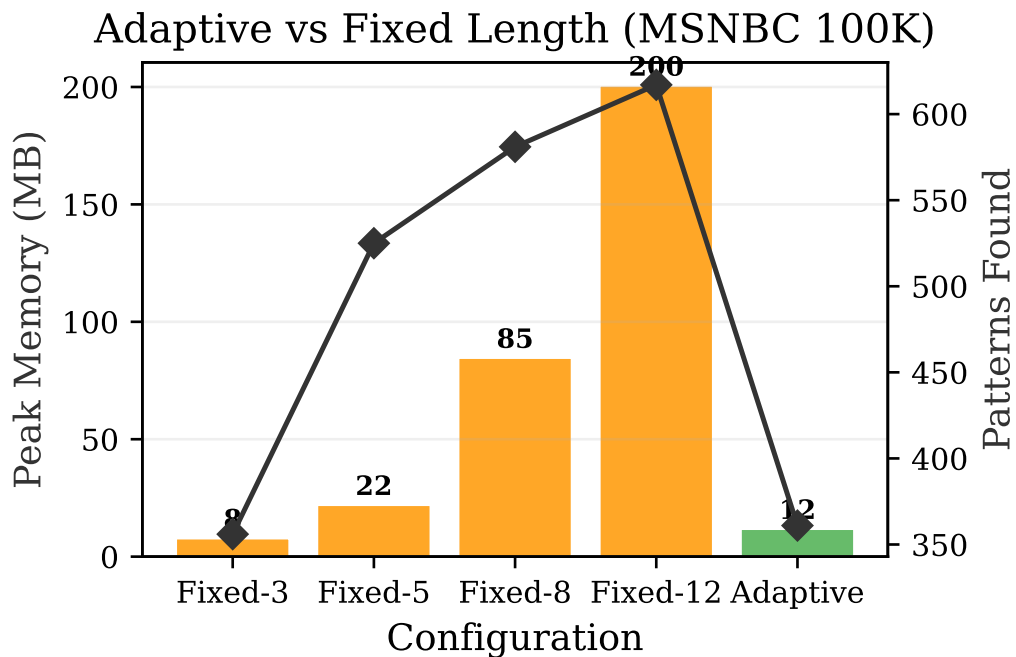
Nhận xét quan trọng: Trên MSNBC, Improved đạt Precision@3 tăng dần theo thời gian (22.2% $\rightarrow$ 50.5%), cho thấy thuật toán cải thiện khi quan sát thêm dữ liệu — đặc trưng quan trọng cho hệ thống khai phá trực tuyến.

5.6 Nghiên cứu cắt bỏ (Ablation Study)

5.6.1 Tác động của Adaptive maxPatternLength

Bảng 5.5: Tác động của Adaptive trên MSNBC (100K sự kiện)

Cấu hình	Patterns	Bộ nhớ	Prec@3	Thời gian
Fixed-3	356	7.7 MB	24.24%	3.31s
Fixed-5	525	21.9 MB	25.76%	5.24s
Fixed-8	581	84.5 MB	25.76%	7.79s
Fixed-12	617	200.4 MB	25.76%	12.23s
Adaptive	361	11.7 MB	24.24%	3.94s



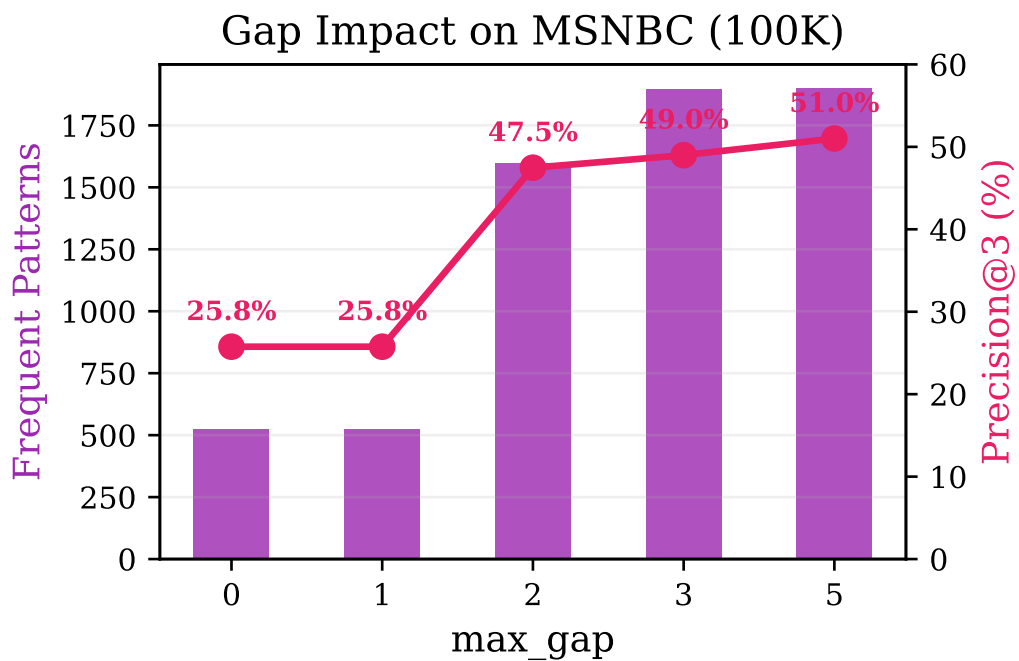
Hình 5.3: Bộ nhớ tiêu thụ theo cấu hình L_{max} : Fixed-12 gấp 17× Adaptive

Kết luận: Bộ nhớ tăng theo hàm mũ khi L_{max} cố định tăng. Adaptive giữ mức bộ nhớ chỉ 11.7 MB (gấp $\frac{1}{17}$ so với Fixed-12), đồng thời vẫn duy trì chất lượng dự đoán tương đương.

5.6.2 Tác động của Gap-Aware Episode Mining

Bảng 5.6: Tác động của Gap Constraint trên MSNBC (100K sự kiện)

max_gap	Patterns	Maximal	Prec@3	Thời gian
0 (tắt)	525	194	25.76%	5.07s
1	525	194	25.76%	13.14s
2	1596	805	47.47%	27.32s
3	1895	1075	48.99%	31.95s
5	1901	1081	51.01%	32.91s



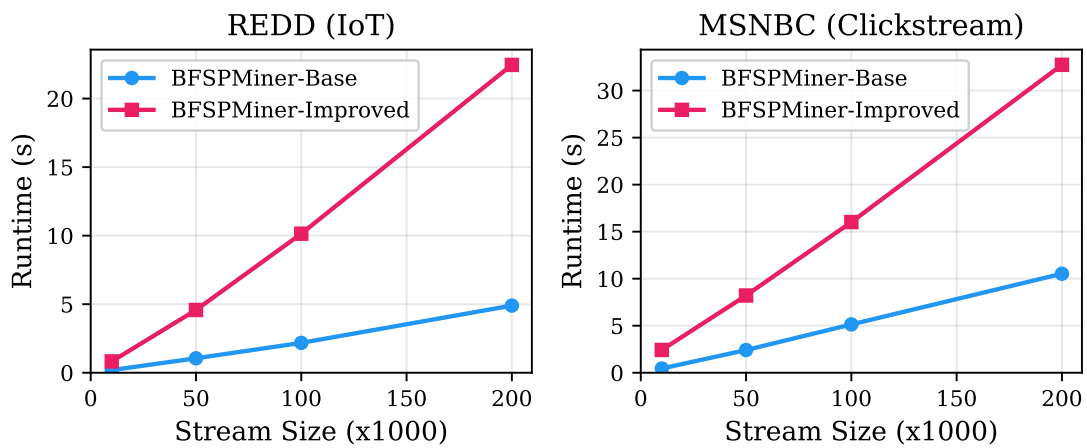
Hình 5.4: Tác động của max_gap trên MSNBC: bước nhảy vọt tại $g = 2$

Kết luận: Bước nhảy vọt xảy ra tại $g = 2$: số mẫu tăng gấp 3 lần ($525 \rightarrow 1596$), Precision@3 gần gấp đôi ($25.76\% \rightarrow 47.47\%$). Tăng gap lên 3–5 cho lợi ích biên giảm dần, khẳng định $g = 2$ là giá trị tối ưu chi phí-hiệu quả.

5.7 Phân tích khả năng mở rộng

Bảng 5.7: Thời gian chạy và bộ nhớ theo kích thước luồng — REDD

Kích thước	Base (s)	Imp (s)	Base (MB)	Imp (MB)
10K	0.19	0.83	0.55	1.0
50K	1.05	4.58	2.51	4.44
100K	2.18	10.14	4.93	9.02
200K	4.90	22.46	9.71	18.68



Hình 5.5: Thời gian chạy theo kích thước luồng: khả năng mở rộng tuyến tính

Cả hai phương pháp thể hiện **khả năng mở rộng tuyến tính**. Improved chậm hơn Base khoảng 4–5× do sinh mẫu gap-aware, nhưng đổi lại +90% mẫu và +26pp Precision@3. Tốc độ xử lý trung bình đạt ~7,100 sự kiện/giây — đủ đáp ứng hầu hết ứng dụng thời gian thực.

Chương 6

Hướng dẫn cài đặt và sử dụng

6.1 Yêu cầu hệ thống

- Python 3.9 trở lên
- pip (trình quản lý gói Python)
- Hệ điều hành: Windows / Linux / macOS
- RAM tối thiểu: 4 GB (khuyến nghị 8 GB cho dữ liệu lớn)

6.2 Cài đặt

```
1 # Clone ăhoc ảgii énn ăm ồngun
2 cd bfspminer-improved
3
4 # àCi đặt ácc ưth ệvin ụph ộthuc
5 pip install -r requirements.txt
```

Listing 6.1: Cài đặt mọi trường

Nội dung file `requirements.txt`:

```
1 psutil>=5.9.0
2 tqdm>=4.64.0
3 pytest>=7.0.0
```

6.3 Chạy Demo cơ bản

```
1 python main.py
```

Listing 6.2: Chạy demo toy stream

Script sẽ khởi tạo BFSPMiner-Improved với Adaptive Length và Episode Gap Extension, xử lý 9 sự kiện mẫu, và in ra:

- Danh sách 5 mẫu phổ biến hàng đầu (pattern, count, support, confidence)
- Dự đoán 2 sự kiện tiếp theo

6.4 Chạy thực nghiệm đầy đủ

```

1 # ạy nhanh (10K events)
2 python main.py --run-experiments --dataset redd --limit 10000
3
4 # ạy đầy đủ (tất cả tập dữ liệu)
5 python main.py --run-experiments --dataset redd --full
6 python main.py --run-experiments --dataset msnbc --full

```

Listing 6.3: Chạy benchmark suite

Kết quả được lưu tại `results/` dưới định dạng JSON.

6.5 Đối chiếu với Java Baseline

```

1 python main.py --compare --dataset redd
2 python main.py --compare --dataset msnbc

```

Listing 6.4: So sánh Python vs Java

6.6 Unit Testing

```

1 pytest tests/ -v

```

Listing 6.5: Chạy kiểm thử

6.7 Cấu trúc mã nguồn

Bảng 6.1: Cấu trúc thư mục mã nguồn

File/Thư mục	Kích thước	Mô tả
core/bfspminer.py	165 dòng	Thuật toán chính BFSPMiner
core/inverted_t0_tree.py	104 dòng	Cây Inverted T_0 (HashMap)
core/adaptive_max_length.py	60 dòng	Cải tiến Adaptive L_{max}
core/episode_extension.py	52 dòng	Cải tiến Gap-Aware
core/config.py	23 dòng	Cấu hình tham số
main.py	88 dòng	Entry point CLI
evaluation/	—	Scripts thực nghiệm
data/	—	Tập dữ liệu REDD, MSNBC
results/	—	Kết quả JSON
figures/	—	Biểu đồ PNG/PDF
tests/	—	Unit tests (pytest)

Chương 7

Kết luận và hướng phát triển

7.1 Kết luận

Bài tập lớn đã thực hiện thành công ba mục tiêu đề ra:

1. Nghiên cứu và cài đặt thuật toán BFSPMiner gốc:

- Triển khai Python hoàn chỉnh, tối ưu cấu trúc dữ liệu (HashMap thay thế ArrayList, tăng tốc tra cứu từ $O(N)$ lên $O(1)$).
- Xác minh 100% chính xác so với bản Java gốc (về số mẫu, support, và luật dự đoán).

2. Đề xuất và cài đặt hai cải tiến:

- **Adaptive maxPatternLength:** Giảm $17\times$ bộ nhớ trên luồng entropy cao (MSNBC), đảm bảo tính khả thi cho triển khai dài hạn.
- **Gap-Aware Episode Mining:** Tăng 84–188% số mẫu phổ biến và gần gấp đôi Precision@3 trên MSNBC ($25.65\% \rightarrow 51.89\%$).

3. Đánh giá thực nghiệm toàn diện:

- Thử nghiệm trên 2 tập dữ liệu thực tế (REDD 458K IoT, MSNBC 424K clickstream).
- 4 bộ thí nghiệm: Overall, Scalability, Ablation Adaptive, Ablation Gap.
- So sánh với 4 baselines: Naive-Random, Naive-Majority, BFSPMiner-Base, BFSPMiner-Improved.

7.2 Đóng góp chính

1. Cơ chế Adaptive maxPatternLength dựa trên entropy và áp lực bộ nhớ — giải quyết trực tiếp hạn chế được tác giả gốc thừa nhận.
2. Mở rộng Gap-Aware Episode Mining cho khung batch-free — lần đầu tiên tích hợp khai phá episode có gap vào thuật toán SPM batch-free.
3. Triển khai Python mã nguồn mở với pipeline thực nghiệm có khả năng tái tạo cao.

7.3 Hướng phát triển

Chúng tôi xác định bốn hướng nghiên cứu trong tương lai:

1. **So sánh với SOTA:** Đánh giá đối chiếu với các thuật toán SPM trên luồng hiện đại (như CPT+, SPAM), xây dựng bộ adapter thống nhất giả định đầu vào.
2. **Cải thiện module dự đoán:** Tích hợp kiến trúc attention-based hoặc neural network vào module dự đoán để nâng cao Precision@3 trên luồng entropy cao.
3. **Song song hóa:** Song song hóa quy trình cập nhật cây T_0 và tích hợp vào hệ sinh thái xử lý phân tán (Apache Flink, Kafka Streams).
4. **Hỗ trợ itemset:** Mở rộng thuật toán để xử lý nhiều item cùng timestamp (ví dụ: nhiều thiết bị bật đồng thời trong IoT).

Phụ lục A

Mã nguồn chính

Toàn bộ mã nguồn được tổ chức trong thư mục `core/` với các file chính sau. Mã nguồn đầy đủ được đính kèm trong thư mục nộp bài.

A.1 `core/config.py` — Cấu hình tham số

```
1 from dataclasses import dataclass
2
3 @dataclass
4 class BFSPMinerConfig:
5     max_pattern_length: int = 5
6     pruning: bool = True
7     delta: int = 1000
8     batch_length: int = 1000
9     alpha: float = 0.05
10    eps: float = 0.1
11    # Extensions
12    enable_adaptive: bool = False
13    enable_gap: bool = False
14    max_gap: int = 1
15    window_size: int = 5
16    # Adaptive Params
17    adaptive_min_len: int = 3
18    adaptive_max_len: int = 15
19    adaptive_check_interval: int = 5000
20    adaptive_memory_threshold_mb: float = 500.0
```

Listing A.1: Cấu hình BFSPMiner

A.2 `core/inverted_t0_tree.py` — Cay Inverted T0

```
1 class TreeObject:
2     __slots__ = ['value', 'parent', 'time_stamps',
3                 'count', 'batch_count', 'tid', 'children']
4     def __init__(self, value, parent, batch_time, time=None):
5         self.value = value
6         self.parent = parent
7         self.count = 1
8         self.batch_count = 1
```

```

9         self.tid = batch_time
10        self.time_stamps = []
11        if time is not None:
12            self.time_stamps.append(time)
13        self.children = {} # HashMap: O(1) lookup
14
15    class InvertedT0Tree:
16        def __init__(self, max_pattern_length):
17            self.max_pattern_length = max_pattern_length
18            self.events = []
19            self.head = TreeObject("root", None, 0)
20            self.head.count = 0
21
22        def add_event(self, event_label, batch):
23            self.events.append(event_label)
24            current_time = len(self.events) - 1
25            pattern_list = [self.events[current_time - i]
26                           for i in range(self.max_pattern_length)
27                           if current_time - i >= 0]
28            visited_nodes = set()
29            self._add_list_to_tree(
30                pattern_list, current_time, batch,
31                self.head, visited_nodes)
32            self.head.count = len(self.events)

```

Listing A.2: Cay Inverted T0 (trich doan)

A.3 core/bfspminer.py — Thuật toán chính

```

1    class BFSPMiner:
2        def __init__(self, config=None, **kwargs):
3            self.config = config if config else BFSPMinerConfig(**kwargs)
4            self.tree = InvertedT0Tree(self.config.max_pattern_length)
5            self.processed_items = 0
6            self.batch = 0
7
8        def feed_item(self, item):
9            if self.config.enable_adaptive:
10                new_len = self.adaptive_length.update(item)
11                self.tree.max_pattern_length = new_len
12            if self.config.enable_gap:
13                patterns = self.episode_extension.generate_patterns_with_gap
14                (
15                    self.tree.events, len(self.tree.events), ...)
16                    self.tree.add_event_with_patterns(item, patterns, self.batch
17                )

```

```

16         else:
17             self.tree.add_event(item, self.batch)
18             self.processed_items += 1
19
20     def get_frequent_patterns(self, min_support):
21         return self.tree.extract_patterns(
22             min_support, max(0, self.processed_items - 1))
23
24     def predict_next(self, k=3, min_support=0.01):
25         # Context matching + confidence ranking
26         ...

```

Listing A.3: BFSPMiner class (trich đoạn)

A.4 core/adaptive_max_length.py — Adaptive Length

```

1 class AdaptiveMaxPatternLength:
2     def __init__(self, config):
3         self.min_len = config.adaptive_min_len      # 3
4         self.max_len = config.adaptive_max_len      # 15
5         self.current_len = config.max_pattern_length
6         self.check_interval = config.adaptive_check_interval
7         self.memory_threshold_mb = config.adaptive_memory_threshold_mb
8         self.event_buffer = []
9
10    def _calculate_entropy(self):
11        counts = {}
12        for e in self.event_buffer:
13            counts[e] = counts.get(e, 0) + 1
14        entropy = 0.0
15        total = len(self.event_buffer)
16        for count in counts.values():
17            p = count / total
18            entropy -= p * math.log2(p)
19        return entropy
20
21    def update(self, item):
22        self.item_count += 1
23        self.event_buffer.append(item)
24        if self.item_count % self.check_interval == 0:
25            memory_mb = psutil.Process().memory_info().rss / (1024*1024)
26            entropy = self._calculate_entropy()
27            if memory_mb > self.memory_threshold_mb:
28                self.current_len = max(self.min_len, self.current_len -
1)
29            elif entropy > 3.0:

```

```

30         self.current_len = max(self.min_len, self.current_len -
31         1)
32         elif entropy < 1.5:
33             self.current_len = min(self.max_len, self.current_len +
34             1)
35             self.event_buffer = []
36         return self.current_len

```

Listing A.4: Adaptive maxPatternLength

A.5 core/episode_extension.py — Gap-Aware Mining

```

1 class EpisodeMiningExtension:
2     def __init__(self, max_gap=1, window_size=5):
3         self.max_gap = max_gap
4         self.window_size = window_size
5
6     def generate_patterns_with_gap(self, events, current_time,
7                                   max_pattern_length):
8         target = events[current_time]
9         patterns = [[target]]
10        start_idx = max(0, current_time - self.window_size + 1)
11        paths = [[target], current_time]
12        valid_patterns_set = set([(target,)])
13        max_combinations = 50 # Hard limit
14
15        for _ in range(max_pattern_length - 1):
16            new_paths = []
17            for current_pattern, last_idx in paths:
18                end_search = max(start_idx, last_idx - self.max_gap)
19                for i in range(last_idx - 1, end_search - 1, -1):
20                    new_pattern = current_pattern + [events[i]]
21                    tup = tuple(new_pattern)
22                    if tup not in valid_patterns_set:
23                        valid_patterns_set.add(tup)
24                        patterns.append(new_pattern)
25                        new_paths.append((new_pattern, i))
26                    if len(new_paths) >= max_combinations:
27                        break
28            paths = new_paths
29        return patterns

```

Listing A.5: Gap-Aware Episode Mining